

Módulo 11 · Engenharia de Software · 3º ano · Aula 1

Spec-Driven Development

Especificar antes de implementar: a especificação como fonte de verdade

Computação 2 · Prof. Afonso Brandão · 04/08/2026

 Especificação Critérios de aceite IA orientada por spec Spec → código

Agenda: 2 Horas de Spec-Driven Development

Estrutura completa da aula

Bloco 1 (30min)

Por quê SDD: problema, motivação, conceitos-chave (SSoT, RF, RNF), qualidade ISO 25010.

Bloco 2 (35min)

Anatomia do Spec: 6 seções (Visão, RF, RNF, API, Schema, Testes). **ADR:** rastreabilidade arquitetural. **Pipeline de elicitação:** 6 estágios.

Bloco 3 (30min)

Modelagem como código: OpenAPI e Gherkin; entidade-relacionamento, diagrama de classes e diagrama de sequência em Mermaid.

Bloco 4 (25min)

SDD e IA: verificação iterativa e demonstração ao vivo sobre um sistema de biblioteca.

 **Objetivo:** Ao final da aula, você será capaz de escrever uma especificação executável com anatomia clara, rastreada em ADRs, que guia desenvolvimento com testes e geração de código assistida por IA.

1. Por Quê Spec-Driven Development?

O problema de começar a implementar antes de especificar

❌ O Problema Comum

Iniciar a implementação sem definição formal do comportamento esperado resulta em:

- Retrabalho constante
- Inconsistência de contratos de API
- Desalinhamento com stakeholders
- Testes incompletos
- Dívida técnica acumulada

✅ A Solução: SDD

Spec-Driven Development é a metodologia onde a especificação técnica é a **única fonte de verdade (Single Source of Truth – SSoT)**:

- Código deriva da spec
- Testes verificam a spec
- Documentação reflete a spec
- IA gera dentro da spec

Conceitos-Chave: Single Source of Truth

A especificação como documento autoritário

Single Source of Truth (SSoT)

Princípio de arquitetura da informação: cada informação relevante reside em **um único artefato autoritário**, do qual todas as demais representações são derivadas. Quando duas versões divergem, a fonte única prevalece; as cópias devem ser reconciliadas com ela, jamais o inverso.

Requisito Funcional (RF)

O que o sistema deve fazer: funções e comportamentos observáveis.

Ex: "O sistema deve permitir registro de usuário por nome e e-mail."

Requisito Não Funcional (RNF)

Como o sistema deve se comportar: qualidade, desempenho, segurança.

Ex: "Registro deve concluir em ≤ 2 segundos sob carga nominal."

SDD vs. TDD

TDD (Test-Driven Development) escreve testes antes do código — fixa correção interna. **SDD** descreve interfaces e contratos antes — fundamenta o TDD. SDD não substitui TDD, o TDD *implementa* a verdade definida pela SDD.

2. Qualidade Não Funcional: ISO 25010:2023

9 características mensuráveis de qualidade de software

As 9 Características ISO 25010

Um modelo internacional que define qualidade de forma objetiva, impedindo que requisitos permaneçam implícitos ou subjetivos:



Desempenho

Tempo de resposta, uso de recursos, capacidade.



Compatibilidade

Coexistência e interoperabilidade com outros sistemas.



Usabilidade

Apreensibilidade, operabilidade, proteção contra erros.



Confiabilidade

Ausência de falhas, disponibilidade, tolerância a falhas.



Segurança

Confidencialidade, integridade, autenticidade.



Manutenibilidade

Modularidade, testabilidade, modificabilidade.



Flexibilidade

Adaptabilidade, escalabilidade, instalabilidade.



Segurança Operacional

Fail-safe, alerta de perigo, recuperação.



Funcionalidade

Correção, completude, propriedade das funções.

3. Os Quatro Pilares de uma Especificação Madura

Componentes essenciais de toda especificação executável

Contrato de API

Rotas, métodos HTTP, códigos de status e payloads de requisição/resposta usando **OpenAPI v3**.

Exemplo: GET /usuarios/{id} retorna {nome, email} ou 404.

Schema de Dados

Tipos, chaves, índices e restrições de integridade no banco de dados.

Exemplo: users.id PK, users.email UNIQUE NOT NULL.

Regras e Cenários

Critérios de aceite em linguagem Gherkin/BDD (Dado/Quando/Então).

Exemplo: "Dado saldo 100, Quando saque 30, Então saldo=70."

Metas de Qualidade

Critérios mensuráveis para cada característica ISO 25010 aplicável.

Exemplo: "Latência $\leq 100ms$ p95, complexidade ciclomática ≤ 5 ."

4. Anatomia de um Spec: Estrutura Completa

Seções obrigatórias e componentes de uma especificação madura

Seção 1: Visão e Escopo

Por que o sistema existe, quem são os usuários, qual problema resolve e o que fica fora de escopo.

"Cotação que reduz a resposta de 24h para <5min."

Seção 2: Requisitos Funcionais

Numerados (RF-001) para que testes e código possam citá-los. Descrição, pré e pós-condições.

"RF-001: cria cotação. Pré: autenticado. Pós: quoteld."

Seção 3: Requisitos Não Funcionais

Numerados (RNF-001), cada um com métrica, limite numérico e instrumento de verificação.

"RNF-001: p95 ≤ 100ms, medida no gateway."

Seção 4: Contrato de API

Rotas, métodos, parâmetros, payloads e códigos de status em OpenAPI 3.1 – com caminho de erro.

POST /api/v1/quotes → 201 {quoteld, total} / 400

Seção 5: Modelo de Dados

Entidades, atributos, tipos, chaves e restrições de integridade – sem citar produto.

Quote {id UUID PK, total Decimal NOT NULL}

Seção 6: Cenários de Teste

Dado/Quando/Então, um por regra de negócio. Cada cenário deve falhar antes da implementação.

"Dado 12 unidades, Quando cotar, Então desconto 5%."

5. ADR: Architecture Decision Record

Rastreabilidade de decisões arquiteturais antes de specs e código

O que é um ADR?

Documento que registra uma decisão arquitetural significativa, seu contexto, opções consideradas, decisão tomada e justificativa. **É independente de tecnologia** – captura o "porquê" de arquitetônico, não o "como" de implementação.

Seções do ADR

- **Título:** ADR-001: Usar padrão de fila para processamento assíncrono
- **Status:** Proposed | Accepted | Deprecated | Superseded
- **Context:** O problema arquitetural
- **Decision:** A escolha feita (sem tecnologia)
- **Rationale:** Por que essa sobre outras
- **Consequences:** Trade-offs e impactos

Exemplo Mínimo

ADR-001: Fila de Processamento

Decision: Processamento assíncrono de cotações via fila.

Rationale: RF-002 exige latência < 2s na resposta. Cálculo de desconto é custoso. Fila desacopla lógica de negócio.

Consequences: +1 componente (fila). RNF: taxa de processamento ≥ 1000 msgs/min.

Por Que ADR Antes de Spec?

ADR → Spec → Implementação. ADR justifica estrutura e padrões que aparecem na spec. Sem ADR, spec parece arbitrária; com ADR, rastreabilidade é clara. Exemplo: "Por que tem fila aqui? Consulte ADR-001."

6. Documentação Técnica: Sequência Completa (Sem Tecnologia)

Problema → ADR → Spec: a jornada antes de código

1 Problema

Contexto de negócio, lacunas, restrições e métricas de sucesso.

Produto e stakeholders. Não é técnico: "o que não funciona e por quê".

2 ADRs

Decisões estruturais: padrões, decomposição, limites de módulo.

Arquitetura. Sem tecnologia: "como estruturamos a solução?".

3 Especificação

RF, RNF, contrato de API, modelo de dados e cenários de aceite.

Arquitetura e time. Ainda sem tecnologia: "qual o comportamento esperado?".

4 Tecnologia (ADR-Tech)

Linguagem, banco e framework, justificados contra a especificação.

Só aqui se escolhe produto. A spec não muda; a tecnologia a implementa.

5 Plano de implementação

Classes, funções e fluxo, abstraídos da sintaxe.

Time com apoio de IA: "quais módulos realizam a spec?".

⚠ Invariante: Problema → ADR → Spec → Tecnologia → Implementação. Nunca pule etapas. Spec não deve conter "use Node.js" ou "use PostgreSQL" — isso vai em ADR-Tech.

7. Pipeline de Elicitação: 6 Estágios

Da análise de negócio à implementação

1 Modelagem de Processos (IDEF0): análise *as-is* — o que a organização faz, lacunas, atores e fluxos.

2 Modelagem Arquitetural (RM-ODP): casos de uso, entidades, diagramas de sequência, 5 viewpoints.

3 Esqueleto do Software: estrutura de diretórios, limites de módulo, Docker Compose, projeto vazio mas estruturalmente completo.

4 Documentos de Especificação (Markdown/YAML): consolidação em formatos legíveis por máquina — **100% autoria humana, IA não participa.**

5 Desenvolvimento Guiado por Testes (TDD): testes que falham inicialmente, consolidando contratos comportamentais (pré-condições, pós-condições, invariantes).

6 Implementação: somente aqui — manualmente ou com IA — o código é gerado para fazer os testes passarem e atender metas de qualidade não funcional.

8. OpenAPI (Swagger) e Gherkin/BDD

Linguagens estruturadas para especificação executável

OpenAPI v3

Especificação padronizada legível por máquina, expressa em YAML ou JSON.

Descreve rotas, métodos HTTP, parâmetros, schemas, códigos de status. Contrato explícito entre produtor e consumidor da API.

```
GET /usuarios/{id} → 200 {nome, email}
| 404
```

Gherkin/BDD

Linguagem estruturada de cenários: Dado/Quando/Então (Given/When/Then).

Legível por humanos e interpretável por ferramentas. Cada cenário = critério de aceite verificável.

```
Dado saldo 100; Quando saque 30; Então
saldo=70.
```

RM-ODP e Viewpoints

Modelo ISO/IEC 10746 que descreve sistemas distribuídos em 5 pontos de vista complementares, evitando mistura de intenção de negócio, modelo de dados e detalhe tecnológico: **Enterprise** (propósito, papéis), **Information** (semântica, estrutura), **Computational** (decomposição funcional), **Engineering** (distribuição), **Technology** (escolhas concretas).

9. Modelagem como Código (*Modeling as Code*)

O modelo é texto versionado no repositório, não imagem anexada

Entende-se por modelagem como código a prática de descrever os modelos do sistema em notação textual — Mermaid, PlantUML, Structurizr DSL — armazenada junto da especificação. O diagrama é derivado do texto por ferramenta; a fonte de verdade é o texto.

Por que texto

- Revisável por *diffe* e sujeito a aprovação em *pull request*.
- Versionado com o código que descreve.
- Divergência entre modelo e implementação torna-se visível.

Três modelos obrigatórios

- **Dados:** entidade-relacionamento (`erDiagram`).
- **Estático:** classes (`classDiagram`).
- **Dinâmico:** sequência (`sequenceDiagram`).

Posição na especificação

- O modelo de dados fundamenta o *schema*.
- O modelo estático delimita responsabilidades.
- O modelo dinâmico define a ordem das interações e os caminhos de erro.

Exemplo conduzido nos próximos slides: sistema de biblioteca — três entidades persistidas e cinco classes de domínio.

10. Modelagem de Dados: Entidade-Relacionamento

Biblioteca — três entidades persistidas, com chaves e cardinalidades explícitas

```
// modelos/dados.mmd
```

```
erDiagram
```

```
LIVRO ||--o{ EMPRESTIMO : origina
```

```
LEITOR ||--o{ EMPRESTIMO : realiza
```

```
LIVRO {
```

```
    uuid id PK
```

```
    text isbn UK
```

```
    text titulo
```

```
    int exemplares
```

```
}
```

```
LEITOR {
```

```
    uuid id PK
```

```
    text email UK
```

```
    bool ativo
```

```
}
```

```
EMPRESTIMO {
```

```
    uuid id PK
```

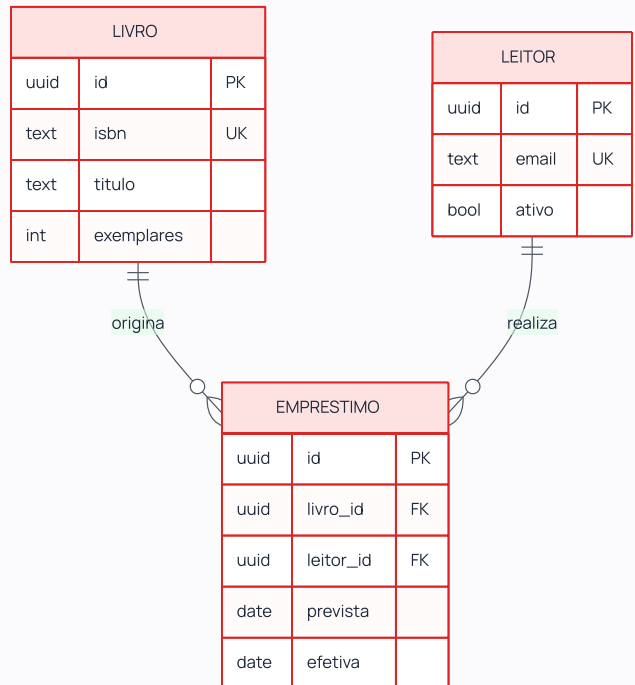
```
    uuid livro_id FK
```

```
    uuid leitor_id FK
```

```
    date prevista
```

```
    date efetiva
```

```
}
```



A cardinalidade `||--o{` fixa a regra: um livro origina zero ou muitos empréstimos.

`efetiva` nula identifica empréstimo em aberto — decisão que deve constar da especificação, não do código.

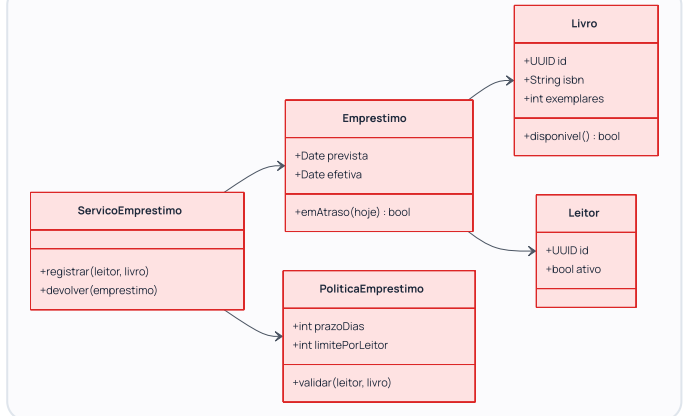
11. Modelagem Estática: Diagrama de Classes

Cinco classes de domínio – estrutura, responsabilidades e dependências

```
// modelos/estatico.mmd
```

classDiagram

```
class Livro {
  +UUID id
  +String isbn
  +int exemplares
  +disponivel() bool
}
class Leitor {
  +UUID id
  +bool ativo
}
class Emprestimo {
  +Date prevista
  +Date efetiva
  +emAtraso(hoje) bool
}
class PoliticaEmprestimo {
  +int prazoDias
  +int limitePorLeitor
  +validar(leitor, livro)
}
class ServicoEmprestimo {
  +registrar(leitor, livro)
  +devolver(emprestimo)
}
ServicoEmprestimo --> PoliticaEmprestimo
ServicoEmprestimo --> Emprestimo
Emprestimo --> Livro
Emprestimo --> Leitor
```



Três classes correspondem às entidades persistidas; duas existem apenas em memória.

`PoliticaEmprestimo` isola as regras de prazo e limite, que mudam por decisão institucional e não por alteração de esquema.

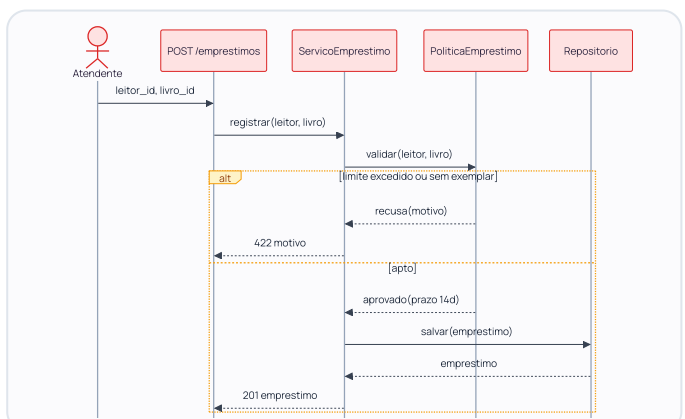
12. Modelagem Dinâmica: Diagrama de Sequência

Registro de empréstimo – ordem das interações e caminho de erro

```
// modelos/dinamico.mmd
```

sequenceDiagram

```
actor A as Atendente
participant API as POST /emprestimos
participant S as ServicoEmprestimo
participant P as PoliticaEmprestimo
participant R as Repositorio
A->>API: leitor_id, livro_id
API->>S: registrar(leitor, livro)
S->>P: validar(leitor, livro)
alt limite excedido ou sem exemplar
P-->>S: recusa(motivo)
S-->>API: 422 motivo
else apto
P-->>S: aprovado(prazo 14d)
S->>R: salvar(emprestimo)
R-->>S: emprestimo
S-->>API: 201 emprestimo
end
```



O bloco `alt` torna o caminho de recusa parte do modelo. Cada ramo corresponde a um cenário Gherkin e a um código de status no contrato OpenAPI.

13. Spec-Driven Development e IA

Como a especificação restringe, codifica e verifica geração de código

💡 Três Papéis da Especificação Diante de Modelos Generativos

A IA gera código sintaticamente correto com facilidade, mas sem alinhamento com requisitos não funcionais quando a instrução é vaga. Uma spec bem estruturada:

📖 Restringe

Limita o espaço de soluções possíveis, evitando decisões arquiteturais improvisadas.

📚 Codifica

Encapsula conhecimento de domínio que o modelo não possui internamente.

✅ Verifica

Define metas de qualidade verificáveis após geração.

Comportamentos que o Modelo Não Infere

Processamento de Eventos Complexos (CEP): análise de fluxos contínuos que correlacionam múltiplos eventos ao longo do tempo (sequências, ausência de eventos, agregações em janelas).

Exemplo: alerta após 3 falhas de autenticação em < 1 minuto. Essas janelas e ordens devem ser definidas explicitamente na especificação.

14. Verificação e Refinamento Iterativo

Como validar que o código atende a especificação

Característica	Estratégia de Verificação
📊 Desempenho	Perfilamento (<i>profiling</i>) de tempo e recursos
⚡ Confiabilidade	Injeção de falhas, teste de fallback
🔒 Segurança	Análise estática, auditoria de código
🔧 Manutenibilidade	Complexidade ciclomática, taxa de duplicação

Ciclo de Verificação: código é gerado → verificado contra metas → desvios → relatório retorna à IA com spec original + instrução de correção → regeneração → reavaliação até que todas as metas sejam atendidas (máx. N iterações).

⚠ **Importante:** Verificação não é uma etapa opcional de revisão manual — é parte integrante do método. Sem verificação, a especificação permanece uma intenção não confirmada.

Ficha Técnica: Spec-Driven Development

Resumo executivo — o que levar para o projeto

Definição

Metodologia na qual a especificação técnica é a **única fonte de verdade (SSoT)**, fundamentando código, testes, documentação e geração por IA.

Quando Usar

Sempre que há múltiplos stakeholders, requisitos não funcionais críticos ou envolvimento de IA na geração de código.

Os 4 Pilares

1. Contrato de API (OpenAPI)
2. Schema de Dados
3. Regras e Cenários (Gherkin)
4. Metas de Qualidade (ISO 25010)

Artefatos Principais

Documento Markdown estruturado, ADR, especificação OpenAPI v3, cenários Gherkin, modelos em Mermaid (dados, classes, sequência), testes automatizados.

Sequência Inviolável (Pipeline 6 Estágios)

1. Processos (IDEF0) → 2. Arquitetura (RM-ODP) → 3. Esqueleto → 4. **Spec (100% humana)** → 5. TDD → 6. Implementação (IA-assistida)

Demonstração ao Vivo

Entrevista com o cliente — plataforma de acervo e empréstimos de uma biblioteca comunitária

Transcrição (trecho)

Cliente: Hoje anotamos em caderno quem levou o quê. Some livro, e ninguém sabe com quem está. Queria um sistema para cadastrar o acervo e controlar os empréstimos.

Analista: Por quantos dias o livro fica com o leitor?

Cliente: Duas semanas, acho. Mas para professor podia ser mais, né?

Analista: Existe renovação ou reserva?

Cliente: Isso eu não tinha pensado. Renovar acho que sim, se ninguém estiver esperando.

Analista: E quando o leitor atrasa?

Cliente: Multa não dá, a biblioteca é comunitária. Talvez bloquear até devolver. Ah, e tem título com dois ou três exemplares iguais. E precisa ser rápido, o computador da recepção é antigo.

Indefinições a resolver antes de codificar

- Prazo fixo ou variável por perfil de leitor — e quem define o perfil.
- Renovação e reserva: existem no escopo? Qual precedência entre elas?
- Consequência do atraso: bloqueio de quê, por quanto tempo, com qual reversão.
- Limite de empréstimos simultâneos por leitor.
- Título e exemplar são a mesma entidade? O empréstimo recai sobre qual deles?
- "Rápido" sem métrica, limite nem instrumento não constitui requisito.

O que será produzido em aula

ADR da decisão estrutural, especificação com RF e RNF mensuráveis, os três modelos em Mermaid — três entidades e cinco classes —, contrato OpenAPI, cenários Gherkin e verificação do código gerado.



Spec-Driven Development

Especificação é contrato. Código é promessa. Testes são prova. IA é executor.

A disciplina de escrever especificações *antes* de qualquer linha de código não é overhead — é o fundamento que transforma ambiguidade em clareza, e ambição em entrega.

Próximas aulas: Modelagem de Dados · Arquitetura de Software · Testing em Escala

Módulo 11 · Engenharia de Software · Aula 1 · Computação 2 · Prof. Afonso Brandão

Sobre este encontro

Spec-Driven Development · 04/08/2026 · Prof. Afonso

OBJETIVO DE APRENDIZAGEM

Ao final do encontro, o estudante deve ser capaz de escrever uma especificação executável com anatomia clara, rastreada em ADRs, que orienta a implementação, os testes e a geração de código assistida por modelos generativos.

ESTRATÉGIA DO ENCONTRO

Exposição em quatro blocos com construção incremental de uma especificação de referência; o encerramento é uma demonstração conduzida pelo professor sobre um sistema de biblioteca, e a aplicação ao projeto do grupo ocorre após a aula, conforme roteiro do material.

ESTRUTURA DO ENCONTRO

1. Bloco 1 (30 min) — Por que SDD: problema, motivação, SSoT, requisito funcional e não funcional, qualidade ISO/IEC 25010
2. Bloco 2 (35 min) — Anatomia da especificação: seis seções, ADR e rastreabilidade, pipeline de elicitação
3. Bloco 3 (30 min) — Modelagem como código: OpenAPI e Gherkin; entidade-relacionamento, classes e sequência em Mermaid
4. Bloco 4 (25 min) — Especificação e IA: verificação iterativa e demonstração ao vivo